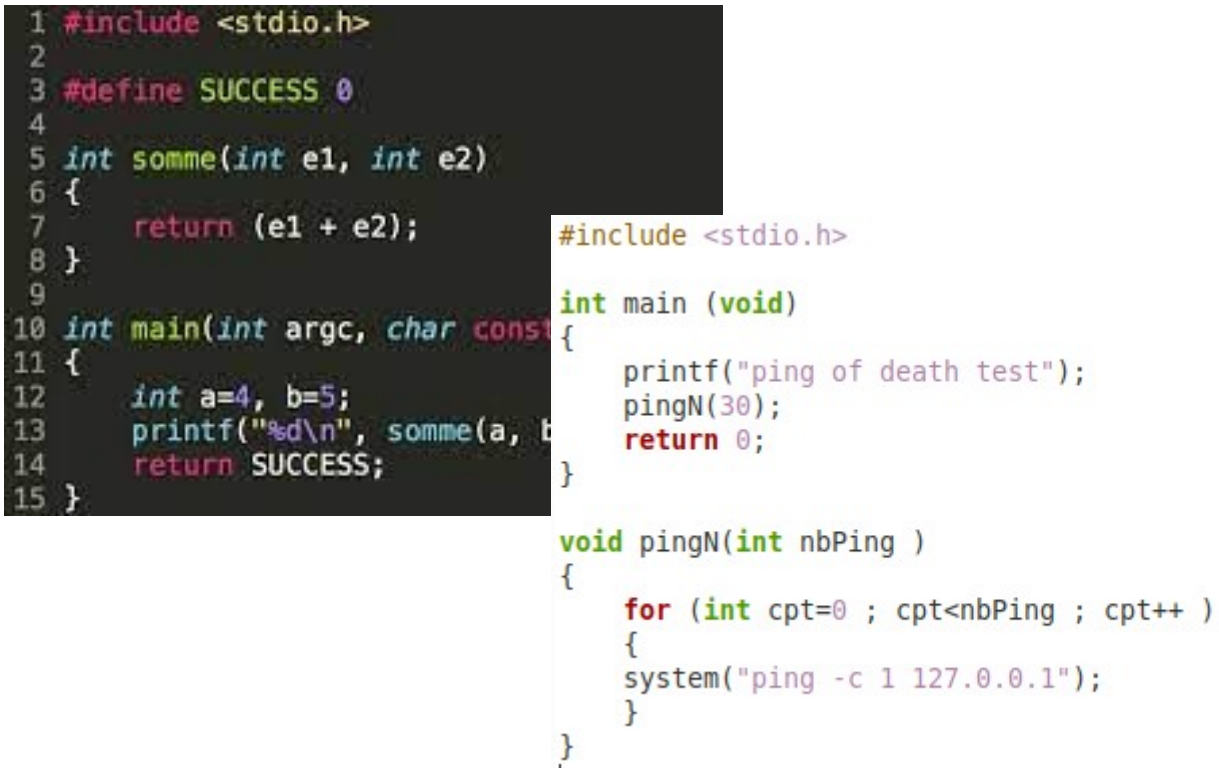
BTS CIEL
PROGRAMMATION EN C



```c
#include <stdio.h>

#define SUCCESS 0

int somme(int e1, int e2)
{
    return (e1 + e2);
}

int main(int argc, char const
{
    int a=4, b=5;
    printf("%d\n", somme(a, b
    return SUCCESS;
}
```

```c
#include <stdio.h>

int main (void)
{
    printf("ping of death test");
    pingN(30);
    return 0;
}

void pingN(int nbPing )
{
    for (int cpt=0 ; cpt<nbPing ; cpt++ )
    {
    system("ping -c 1 127.0.0.1");
    }
}
```

1 LE LANGAGE C

1.1 Introduction

Le seul langage compréhensible par le microprocesseur est appelé langage machine. Ce langage est très difficile à utiliser et on préfèrera employer des langages dits de « haut niveau » tel que le C. Pour écrire un programme en C on aura recours à un logiciel de « traduction » appelé compilateur.

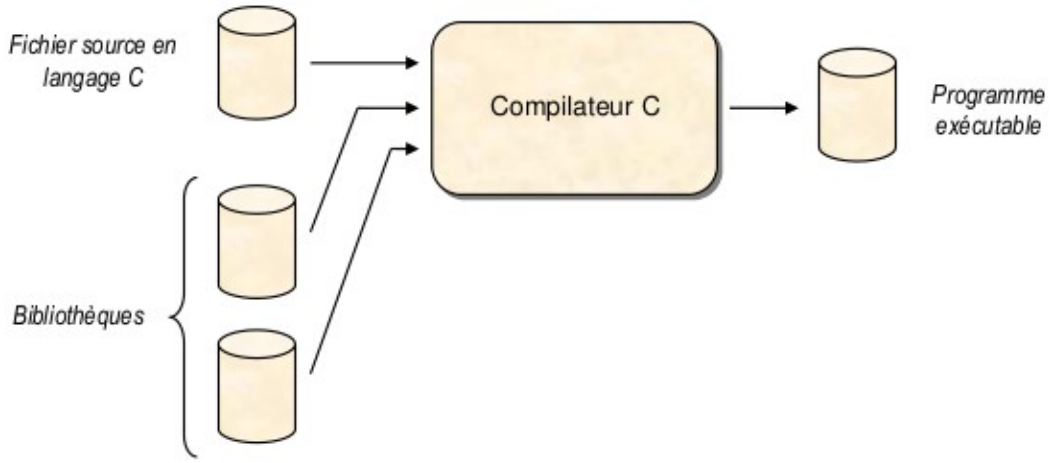


Figure 1: Compilateur

Le fichier contenant le programme écrit en C est appelé « Source ». Le compilateur reçoit en entrée le fichier source ainsi que des bibliothèques de fonctions, et produit le programme exécutable par le microprocesseur.

1.2 Types de données

Tous les systèmes à microprocesseurs sont amenés à traiter des données, par exemple, dans une machine à laver, le programme doit gérer la température de l'eau, la vitesse de rotation du tambour, etc. Le microprocesseur ne peut manipuler que des données binaires le plus souvent de huit bits, c'est-à-dire des nombres pouvant varier de 0 à 255, il ne connaît donc ni les nombres négatifs, ni les lettres de l'alphabet, ni les nombres réels, etc. Pour faciliter le travail de programmation, le compilateur C prend en charge plusieurs types de données :

Type	Signification	Taille en mémoire	Plage de variation
int	Nombre entier avec signe	1 à 4 octets	De [- 2^7 ; 2^7-1] à [-2^{31} à 2^{31}-1]
unsigned int, uint	Nombre entier sans signe	1 à 4 octets	De [0 ; 2^8-1] à [0 à 2^{32}-1]
long	Entier long avec signe	2 à 8 octets	De [- 2^{15} ; 2^{15}-1] à [-2^{63} à 2^{63}-1]
unsigned long	Entier long sans signe	2 à 8 octets	De [0 ; 2^{16}-1] à [0 à 2^{64}-1]
float	Nombre à virgule flottante	4 octets	≈1,17.10^{-38} à 6,80.10^{38}
char	Caractère alphanumérique	1 octet	0 à 255 (cf table ascii)

A noter : les types int et uint peuvent varier en taille en fonction des architectures et des compilateurs. Pour s'en convaincre, il est possible d'utiliser la fonction sizeof() qui renvoie la taille du type ou de la variable en octets :

Exemple sur PC:

```
printf( "taille du type int: %d\n", sizeof( int ) );
```

→ taille du type int: 4

Exemple sur arduino Uno :

```
Serial.print("taile du type int :");
Serial.println(sizeof(int));
```

→ taille du type int: 2

Ces différences peuvent nuire à la portabilité du code et créer des bugs de débordements (overflow) pouvant s'avérer graves et difficiles à détecter. C'est pourquoi il existe en C/C++, des types dont la taille est explicite, dont l'utilisation est recommandée :

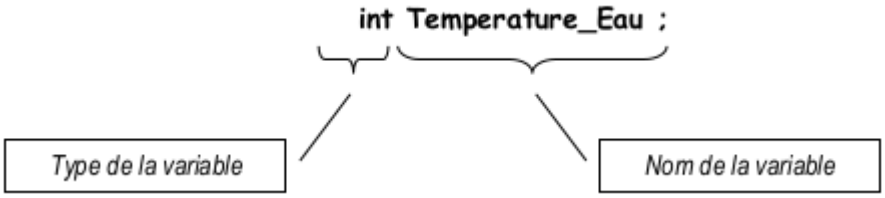
int8_t, int16_t, int32_t, int64_t
désignent respectivement des entiers signés sur 8, 16, 32 et 64 bits.

uint8_t, uint16_t, uint32_t, uint64_t
désignent respectivement des entiers non signés sur 8, 16, 32 et 64 bits.

1.3 Les variables

1.3.1 *Déclaration et initialisation*

Une variable est un objet repéré par son nom, destiné à contenir des données qui pourront être modifiées durant l'exécution du programme. En langage C toute variable doit posséder un type et avoir été déclarée avant d'être utilisée. Pour déclarer une variable on indique tout d'abord son type, puis son nom



Il est possible de déclarer, en une seule fois, plusieurs variables de même type en séparant leurs noms par des virgules :

```
int Temperature_Eau , Consigne_Temperature ;
```

Il est conseillé d'initialiser les variables lors de leur déclaration, c'est-à-dire de leur affecter une valeur. Exemple : on crée une variable de type entier appelée Temperature_Eau et on lui affecte la valeur 25 :

```
int Temperature_Eau = 25;
```

1.3.2 *Affectation*

Une fois la variable déclarée, il devient possible de lui affecter une valeur à tout moment en utilisant l'opérateur '='.

```
Temperature_Eau = 32;
Consigne_Temperature = 50;
```

Les valeurs numériques peuvent être exprimées en décimal, c'est le cas par défaut, mais aussi en hexadécimal ou en binaire en employant des caractères spéciaux.

```
LED = 0b00110111 ;    // Valeur en binaire
REG_CONF = 0x3F ;     // Valeur en hexadécimal
```

1.4 Les constantes

Les constantes se déclarent et s'initialisent comme des variables, mais elles ne pourront pas être modifiées ultérieurement par le programme. Pour déclarer une constante on utilise le mot clé const.

```
const float pi=3.14159 ;
```

1.5 Les opérateurs

Les opérateurs sont des symboles qui permettent de manipuler les variables, c'est-à-dire d'effectuer des opérations, de les évaluer, etc.

1.5.1 *Les opérateurs de calcul*

Ils permettent de modifier le contenu des variables en utilisant des opérations mathématiques

Opérateur	Dénomination	Effet	Exemple	Résultat (avec x valant 7)
=	Affectation	Affecte une valeur	x=3	3
+	Addition	Additionne deux valeurs	x=x+2	9
*	Multiplication	Multiplie deux valeurs	x=3*2	6
/	Division	Divise deux valeurs	x=10/2	5
%	Modulo	Reste de la division	x=7%2	1

Figure 2: Les opérateurs de calcul

1.5.2 *Les opérateurs d'incrémentation / décrémentation*

Ces opérateurs permettent d'ajouter ou de retrancher 1 à une variable. Ce type d'opérateur est très utile pour la réalisation de boucles.

Opérateur	Dénomination	Effet	Exemple	Résultat (avec x valant 7)
++	Incrémentation	Ajoute 1 à la valeur de la variable.	x++	8
--	Décrémentation	Retranche 1 à la valeur de la variable.	x--	6

Figure 3: Les opérateurs d'incrémentation décrémentation

1.5.3 *Les opérateurs logiques bits à bits*

Ce type d'opérateur permet de manipuler chaque bit d'une donnée. Ils sont extrêmement utiles pour gérer les opérations d'entrée sortie.

Opérateur	Dénomination	Effet	Exemple	Résultat (avec x valant 7)
&	ET	Effectue un ET logique entre deux bits de même poids.	5&3 (101)&(011)	1 (001)
|	OU	Effectue un OU logique entre deux bits de même poids.	5|3 (101)|(011)	8 (111)
^	OU Exclusif	Effectue un XOR logique entre deux bits de même poids.	5^3 (101)^(011)	6 (110)
~	Complément	Complémente chaque bit	~5 ~(101)	2 (010)
<<	Décalage à gauche.	Décale les bits vers la gauche.	5<<1 (101)<<1	10 (1010)
>>	Décalage à droite.	Décale les bits vers la droite.	5>>1 (101)>>1	2 (10)

Figure 4: Les opérateurs logiques bit à bit

1.5.4 *Les opérateurs de comparaison*

Les opérateurs de comparaisons permettent d'effectuer des tests sur les variables afin de prendre des décisions (voir § 3.2.4). En langage C, une comparaison réussie retournera la valeur '1', qui correspond à l'état vrai (true). A l'inverse, une comparaison qui échoue retournera la valeur '0', qui correspond à l'état faux (false).

Exemple : La proposition 5 < 3 retournera la valeur 0 (faux), tandis que la proposition 5 > 3 retournera la valeur 1 (vrai).

Ces opérateurs permettent de tester des variables numériques quelconques

Opérateur	Dénomination	Syntaxe	Fonction
>	Strictement supérieur	a > b	Retourne '1' si a est supérieur à b
>=	Supérieur ou égal	a >= b	Retourne '1' si a est supérieur ou égal à
<	Strictement inférieur	a < b	Retourne '1' si a est inférieur à b
<=	Inférieur ou égal	a <= b	Retourne '1' si a est inférieur ou égal à
==	Egal	a == b	Retourne '1' si a est égal à b
!=	Différent	a != b	Retourne '1' si a est différent de b

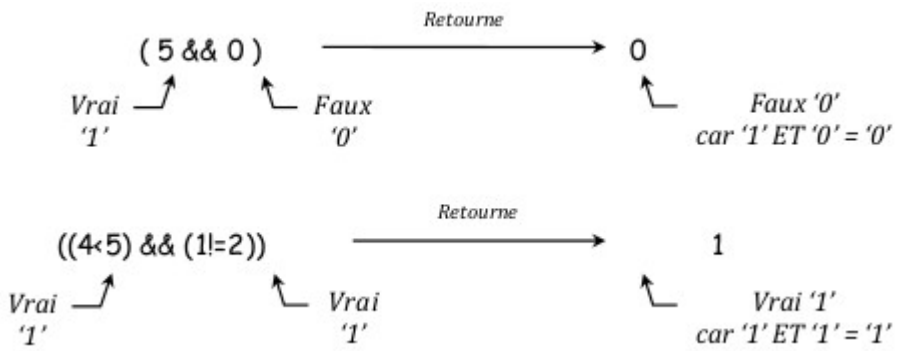
Figure 5: Les opérateurs de comparaison aruthmétiques

Les opérateurs booléens permettent de tester des conditions booléennes, c'est-à-dire que les variables sont considérées comme pouvant être soit vraies (valeur différente de 0), soit fausses (valeur égale à 0).

Opérateur	Dénomination	Syntaxe	Fonction
&&	ET logique	a && b	Retourne 1 si a ET b sont vrais
||	OU logique	a || b	Retourne 1 si a OU b sont vrais
!	NON logique	! a	Retourne 1 si a est faux Retourne 0 si à est vrai

Figure 6: Les opérateurs de comparaison booléens

Exemple :



1.6 Structures de contrôle

Les structures de contrôle permettent au programme :

- De prendre des décisions, se sont alors des structures conditionnelles.
- De répéter certaines instructions sous certaines conditions, ce sont alors de structures de boucles.

1.6.1 *Instructions et blocs d'instructions*

En langage C le point-virgule est utilisé pour séparer les instructions. Une instruction est une expression suivit d'un point-virgule.

Exemple :

```
delta = (b*b)-(4*a*c);
```

Il est possible de regrouper plusieurs instructions pour créer un ensemble appelé bloc d'instructions. Pour cela les instructions formant le bloc sont placées entre des accolades et séparées par des points virgules..

Exemple :

Début du bloc d'instructions →

```
{
  delta = (b*b) - (4*a*c);
  r_prime = (-b + sqr(delta)) / (2*a);
  r_seconde = (-b - sqr(delta)) / (2*a);
}
```

← Début du bloc d'instructions

1.6.2 *L'instruction 'if'*

Elle permet d'effectuer une instruction ou un bloc d'instruction lorsqu'une condition est réalisée.

Structure ' if ' avec un bloc d'instruction :

```
if ( Condition )
{
  Instruction_1;
  Instruction_2;
}
```

Structure ' if ' avec une seule instruction :

```
if ( Condition ) Instruction ;
```

Exemple :

```
if (temperature > 37 )
  printf( "Vous êtes malade !" ) ;
```

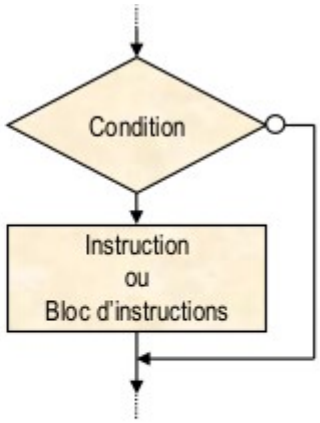


Figure 15 – L'instruction if

1.6.3 *L'instruction ' if else '*

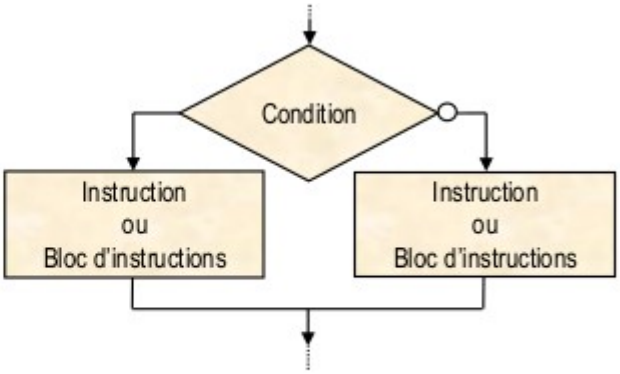
Elle permet de choisir entre deux instructions, ou deux blocs d'instructions, lequel va-t-être exécuté en fonction d'une condition.

```
if ( Condition )

 {
   Instruction_1;
   Instruction_2;
 }

else

 {
   Instruction_3;
   Instruction_4;
 }
```



Si la condition est vérifiée (vraie), les instructions Instruction_1 et instruction_2 seront exécutées ; sinon se sont les instructions Instruction_3 et Instruction_4 qui le seront.

Exemple :

```
if ( temperature > 37 )
    printf( "Vous êtes malade !" ) ;
else
    printf("Vous n'êtes pas malade.");
```

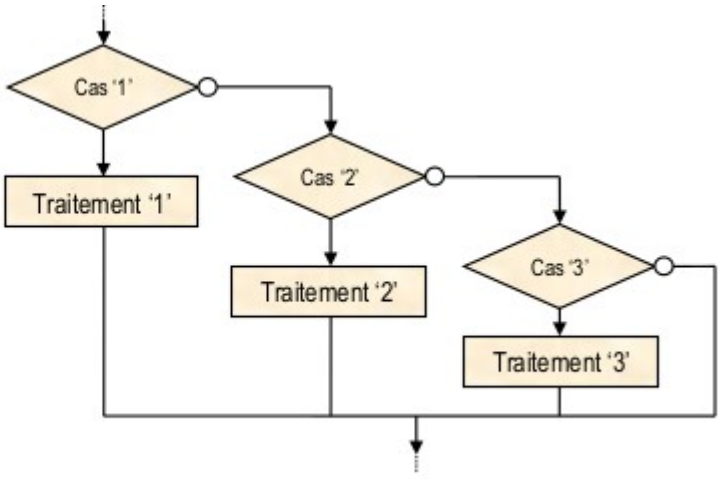
1.6.4 *L'instruction ' switch case '*

Elle permet de sélectionner, en fonction de la valeur d'une variable, quelle instruction, ou quel bloc d'instruction, sera exécuté.

```
switch ( Variable )
 {
  case Valeur_1 :
   Traitement_1;
   break;

  case Valeur_2 :
   Traitement_2;
   break;

  case Valeur_3 :
   Traitement_3;
   break;
 }
```



1.6.5 *La boucle ' while '*

Elle permet de répéter une instruction ou un bloc d'instruction tant qu'une condition est réalisée.

Boucle while avec un bloc d'instruction :

```
while ( Condition de continuation )
 {
  Instruction_1;
  Instruction_2;
  Instruction_3;
 }
```

Boucle while avec une seule instruction :

```
while ( Condition de continuation )

  Instruction_1;
```

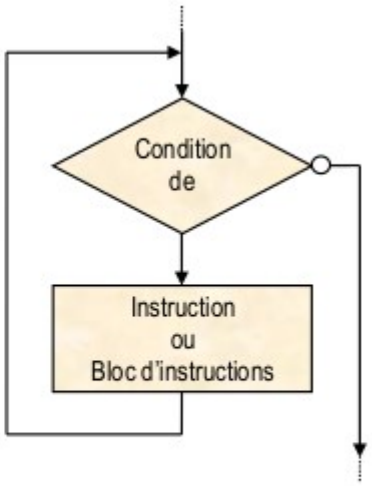
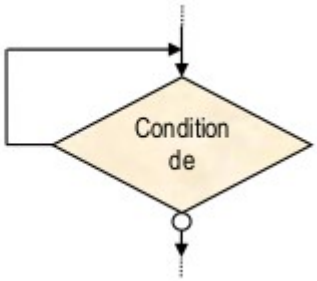


Figure 18 – La boucle while

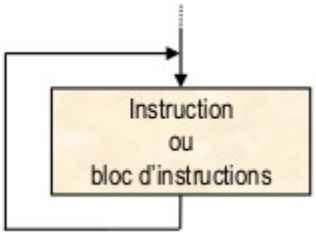
- Cas particulier : La boucle while vide : Lorsque la boucle while ne contient aucune instruction, le programme « attend » que la condition ne soit plus réalisée pour se poursuivre.

```
while ( Condition de continuation ) ;
```



- Cas particulier : La boucle while infinie : Il s'agit d'une boucle while dans laquelle la condition est remplacée par la valeur 1, dans ce cas la condition est toujours vraie et la boucle se répète indéfiniment

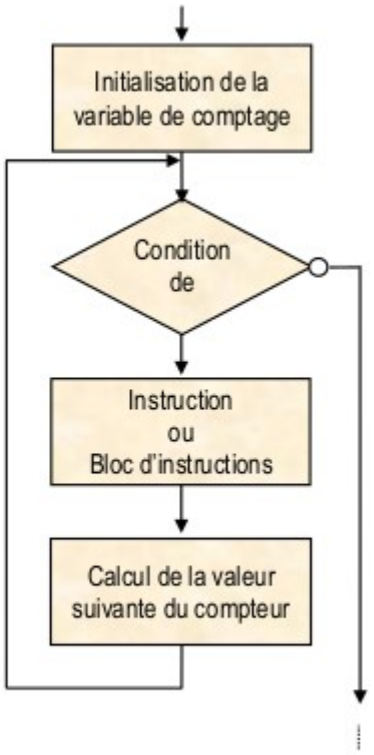
```
while( 1 )
{
 Instruction_1;
 Instruction_2;
 Instruction_3;
}
```



1.6.6 *La boucle ' for '*

Cette structure permet de répéter une instruction ou un bloc d'instruction en contrôlant exactement le nombre d'itérations (on appelle itération une exécution de l'instruction ou du bloc).

```
for (Compteur = Valeur_Initiale ; Condition ; Incrémentation )
 {
   Instruction_1 ;
   Instruction_2 ;
 }
```



Exemple 1 :

Cette boucle affiche les nombres entiers de 0 à 9.

Le compteur Ctr est initialisé à la valeur 0, il sera incrémenté à chaque itération de la boucle, et celle-ci se poursuivra tant que Ctr restera strictement inférieur à 10.

```
for(Ctr = 0 ; Ctr < 10 ; Ctr++ )
 {
  printf ( "Le compteur vaut %d \r\n", Ctr ) ;
 }
```

Exemple 2 :

```
for(Ctr = 5 ; Ctr <= 8 ; Ctr++ )
```

`Ctr` prendra successivement les valeurs : 5,6,7,8 ; les instructions présentes dans la boucle seront donc exécutées 4 fois.

Exemple 3 :

```
for(Ctr = 15 ; Ctr > 12 ; Ctr-- )
```

`Ctr` prendra successivement les valeurs : 15,14,13 ; les instructions présentes dans la boucle seront donc exécutées 3 fois.

Exemple 4 :

```
for(Ctr = 2 ; Ctr < 15 ; Ctr = Ctr +2 )
```

`Ctr` prendra successivement les valeurs : 2,4,6,8,10,12,14 ; les instructions présentes dans la boucle seront donc exécutées 7 fois.

1.7 Les fonctions

Une fonction est un sous-programme qui permet d'effectuer un ensemble d'instruction par simple appel de la fonction dans le corps principal du programme. Cela est utile lorsqu'un programme doit effectuer plusieurs fois la même tâche.

Par exemple, pour faire clignoter une LED, il faut effectuer deux temporisations. Pour ne pas écrire deux fois les mêmes instructions on pourra avoir recours à une fonction.

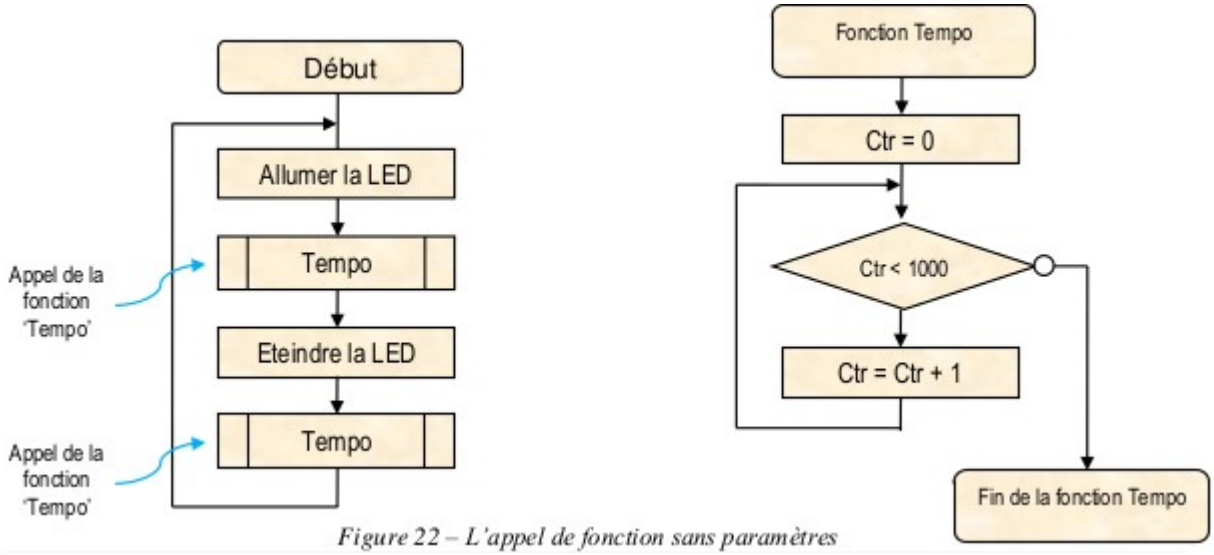


Figure 22 – L'appel de fonction sans paramètres

```
// Fonction de temporisation
void Tempo( void )
{
    int Ctr ;
    for( Ctr=0; Ctr<1000; Ctr++);
}

// Corps principal du programme
void loop()
{
    LED = 1;        // Allume la Led
    Tempo();        // Attend
    LED = 0;        // Eteind la Led
    Tempo();        // Attend
}
```

1.7.1 *Fonction avec passage de paramètres*

Il est possible de passer un ou plusieurs paramètres à une fonction lors de son appel. Par exemple, on peut passer un paramètre à la fonction Tempo pour choisir la durée de temporisation lors de l'appel de la fonction.

Dans l'exemple ci-dessous, la fonction Tempo reçoit un paramètre appelé Duree, de type int. Le programme principal appelle deux fois la fonction, le premier appel se fait avec la valeur 100 en paramètre, et le second appel se fait avec la valeur 1000.

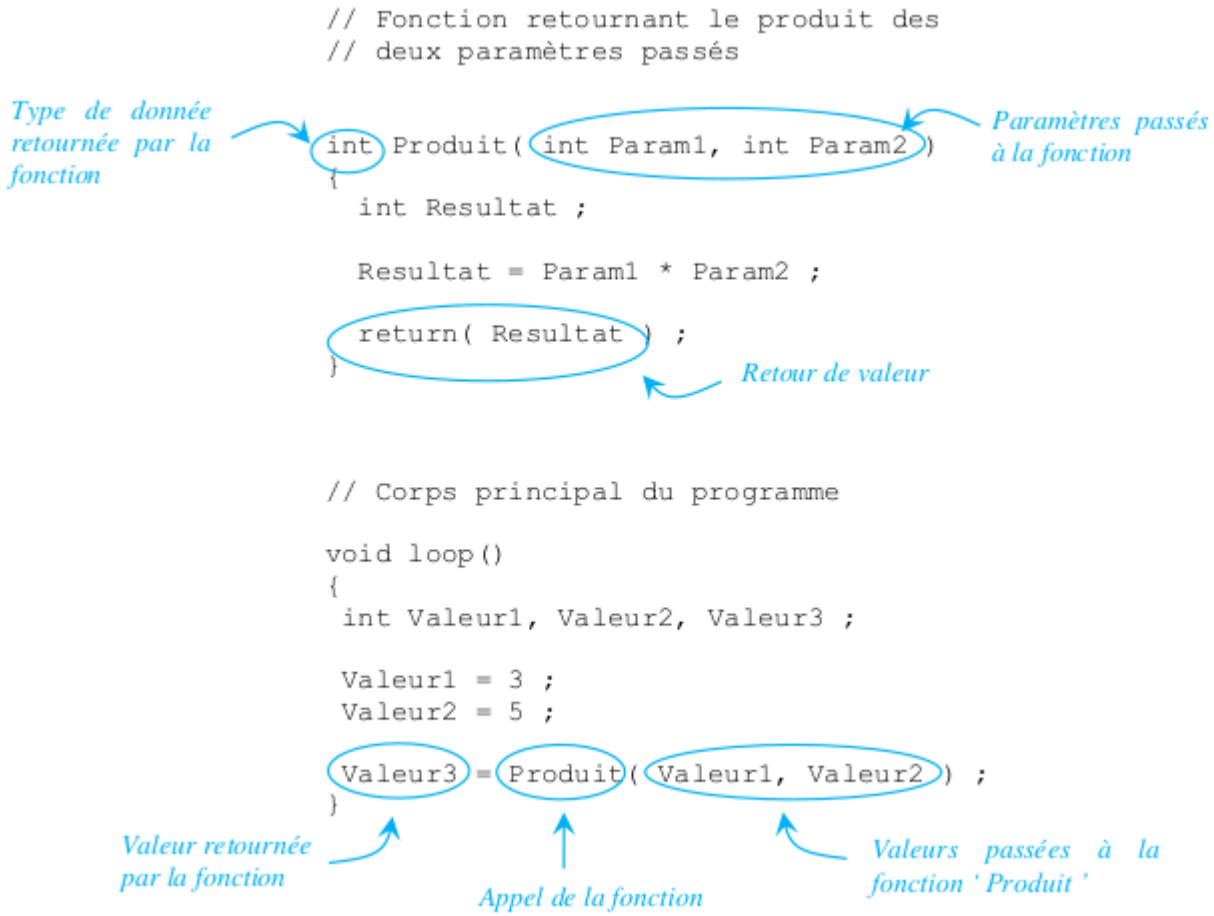
Dans ce nouveau programme la Led restera éteinte dix fois plus de temps qu'elle ne restera allumée.

```
// Fonction de temporisation recevant un paramètre
void Tempo( int Duree )
{
    int Ctr ;
    for( Ctr=0; Ctr<Duree; Ctr++);
}

// Corps principal du programme
void loop()
{
    LED = 1;                // Allume la Led
    Tempo( 100 );  // Appel de Tempo avec 100 en paramètre
    LED = 0;                // Eteind la Led
    Tempo( 1000 );          // Appel de Tempo avec 1000 en paramètre
}
```

1.7.2 *Fonction retournant une valeur*

Une fonction peut retourner une valeur au programme qui l'a appelé. Il faut pour cela préciser, lors de la déclaration de la fonction, le type de donnée renvoyée. Dans le corps de la fonction, le retour de valeur s'effectue grâce à l'instruction return.



La fonction Produit prend en entrée les paramètres Param1 et Param2, calcule le produit des deux et retourne le résultat. Le programme appelle la fonction Produit avec les valeurs 3 et 5, et stocke le résultat retourné dans la variable Valeur3. Après l'exécution du programme la variable Valeur3 contiendra la valeur 15.

2 SOURCES / POUR EN SAVOIR PLUS

→ Cours de Bruno Masi (prof.masi.fr)

→ The C Programming Language, Livre de Brian Kernighan et Dennis Ritchie

→ Pour s'exercer à la programmation et préparer des projets sur les cartes arduino (et arduino compatibles)

- Site arduino, section reference arduino.cc/reference
- Site arduino, section software : arduino.cc/en/software
- Simulateur arduino en ligne, sans inscription : wokwi.com

→ Pour s'exercer à la programmation pour la plupart des langages courants : replit.com

→ Pour vous entraîner hors ligne (et être plus efficace) le mieux est d'installer un IDE (environnement de développement intégré) sur votre ordinateur.

On peut citer CodeBlocks, Codium , Eclipse , et bien d'autres… Le choix du langage et de l'IDE dépend de nombreux paramètres, comme la cible matérielle (microcontrôleur type arduino, ordinateur, ou autre..) sur laquelle va s'exécuter votre programme et son but (robot, jeu, automate, etc.).